

Compilador HULK

Reporte de Proyecto Final

Asignatura: Compilacion / Lenguajes de Programacion

Facultad de Matematica y Computacion

Universidad de La Habana

Ciencias de la Computacion — Tercer Ano

6 de junio de 2026

Resumen

Este documento describe el diseno, la arquitectura y la implementacion de un compilador completo para el lenguaje **HULK** (Havana University Language for Kids), desarrollado enteramente en el lenguaje de programacion **Rust** con backend LLVM 17 mediante la biblioteca *inkwell*. El compilador cubre todos los requerimientos obligatorios de la especificacion HULK (secciones 1 a 16.8, incluyendo verificacion de tipos) y extiende el lenguaje con un **sistema de vectores con expresiones generadoras**, una extension sintactica y semantica con base en la teoria de comprensiones de listas presentes en lenguajes como Haskell y Python. El documento incluye una descripcion detallada de cada fase del compilador, las decisiones de diseno, comparativas con otros lenguajes para las extensiones propuestas, y una evaluacion de la cobertura de la suite de pruebas oficial.

Tecnologias utilizadas: Rust 1.77+, LLVM 17, inkwell 0.4, LALR(1) artesanal, NFA por construccion de Thompson.

Lineas de codigo Rust: $\approx 8\,000$ LOC fuente + $\approx 2\,000$ LOC de pruebas.

Tests automatizados: 384 pruebas en `cargo test` + suite oficial del profesor.

Índice

1. Introduccion	4
1.1. Objetivos del proyecto	4
1.2. Estructura del compilador	4
1.3. Eleccion del lenguaje: Rust	5
2. Analisis Lexico	6
2.1. Arquitectura: Lexer basado en NFA	6
2.2. Conjunto de tokens	6
2.3. Procesamiento de secuencias de escape	7
2.4. Reporte de errores lexicos	7
3. Analisis Sintactico	8
3.1. Parser LALR(1) implementado desde cero	8
3.2. Jerarquia de la gramatica y precedencia de operadores	8
3.3. Arbol de Sintaxis Abstracta (AST)	9
4. Analisis Semantico	10
4.1. Verificacion de tipos en dos pasadas	10
4.2. Sistema de tipos	10
4.3. Funciones y constantes predefinidas	11
4.4. Reporte de errores semanticos	11
5. Generacion deCodigo	11
5.1. LLVM IR mediante inkwell	11
5.2. Representacion de valores (CgValue)	12
5.3. Representacion de objetos en memoria	12
5.4. Vtables y despacho virtual	13
5.5. Type tags DFS y operador <code>is</code>	13
5.6. Operador <code>as</code> – downcast con verificacion en runtime	14
5.7. Llamada a <code>base()</code>	14
5.8. Pipeline de optimizacion	14
6. Biblioteca de Runtime	14
6.1. Layout de vectores en memoria	15
6.2. Pipeline AOT completo	15
7. Extension al Lenguaje: Vectores y Expresiones Generadoras	16
7.1. Motivacion	16
7.2. Sintaxis de la extension	16
7.2.1. Vectores explicitos	16
7.2.2. Indexacion	17
7.2.3. Expresiones generadoras	17
7.2.4. Tamano y metodo <code>size</code>	17
7.2.5. Tipo <code>[T]</code> en el sistema de tipos	17
7.3. Semantica de evaluacion: estrategia <i>eager</i>	18
7.3.1. Comparacion con Python	18

7.3.2.	Comparacion con Haskell	18
7.3.3.	Comparacion con Scala	19
7.3.4.	Comparacion con C++	19
7.3.5.	Resumen comparativo	20
7.4.	Implementacion del codegen para vectores	20
7.4.1.	Vector explicito	20
7.4.2.	Expresion generadora	20
7.5.	El tipo Range como iterable	21
7.6.	Justificacion de las decisiones de diseno	21
7.6.1.	Por que eager y no lazy?	21
7.6.2.	Por que como separador?	22
7.6.3.	Por que vectores de tamano fijo?	22
8.	Cobertura de la Especificacion HULK	22
8.1.	Ejemplos de programas HULK completos	23
8.1.1.	Herencia y polimorfismo	23
8.1.2.	Protocolos	24
8.1.3.	Extension: generadores	24
9.	Pruebas y Evaluacion	24
9.1.	Suite de pruebas automatizadas	24
9.2.	Suite oficial del profesor	25
9.3.	Contrato de errores	25
10.	Decisiones de Diseno	25
10.1.	LALR(1) artesanal vs generador externo	25
10.2.	Backend LLVM vs interprete vs bytecode propio	26
10.3.	Vtables vs tagged union vs boxed types	26
10.4.	Enlace dinamico de LLVM vs estatico	26
11.	Limitaciones Conocidas y Trabajo Futuro	26
11.1.	Limitaciones actuales	26
11.2.	Trabajo futuro	27
12.	Conclusiones	27

1. Introduccion

HULK es un lenguaje de programacion disenado con fines educativos en la Facultad de Matematica y Computacion de la Universidad de La Habana. Es un lenguaje orientado a expresiones, estaticamente tipado, con soporte para programacion orientada a objetos (herencia simple, polimorfismo, protocolos) y construcciones funcionales (funciones de primera clase, expresiones generadoras).

1.1. Objetivos del proyecto

El objetivo principal es implementar un compilador de produccion para HULK que:

1. Cubra **todos** los features obligatorios de la especificacion (hasta la seccion 16.8, verificacion de tipos incluida).
2. Genere **codigo nativo** x86-64 mediante LLVM 17 para obtener ejecutables eficientes que no requieran un interprete en tiempo de ejecucion.
3. Implemente una **extension propia** al lenguaje que modifique tanto la sintaxis como la semantica, con fundamentacion teorica y comparativa respecto a otros lenguajes.
4. Sea completamente desarrollado en **Rust**, aprovechando las garantias de seguridad de memoria del lenguaje anfitrión.

1.2. Estructura del compilador

El compilador sigue el diseno clasico de compiladores en fases encadenadas:

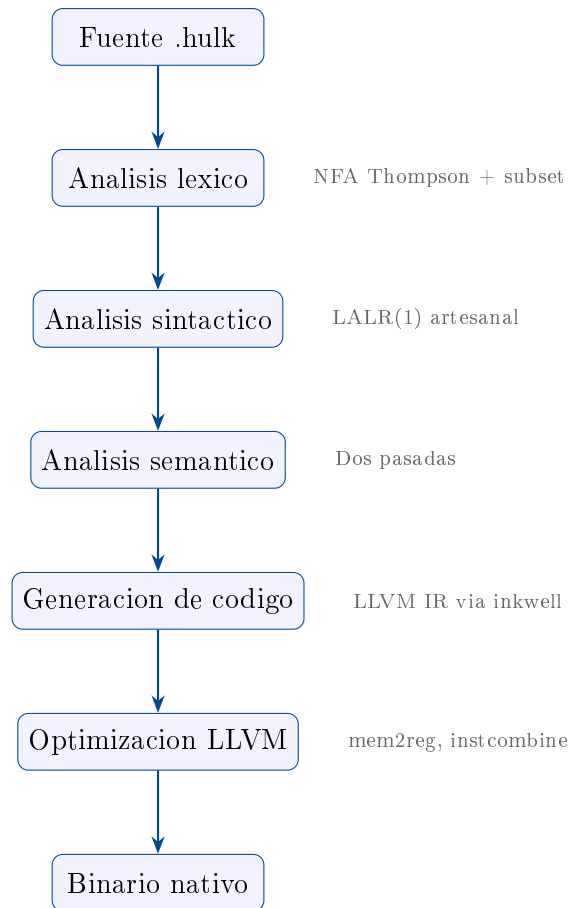


Figura 1: Pipeline de compilacion de HULK.

Cada fase es un modulo Rust independiente con interfaces bien definidas, lo que facilita el desarrollo incremental, las pruebas unitarias y el mantenimiento.

1.3. Eleccion del lenguaje: Rust

La especificacion permite C, C++ o Rust. Se eligio Rust por tres razones principales:

- **Seguridad de memoria sin GC:** el compilador maneja estructuras de datos complejas con muchas referencias cruzadas (tablas de simbolos, jerarquias de tipos, tabla de parseo LALR). Rust garantiza ausencia de dangling pointers y data races en tiempo de compilacion, eliminando una clase entera de bugs dificiles de depurar.
- **Bindings seguros a LLVM:** la biblioteca *inkwell* proporciona una API de alto nivel que envuelve la C API de LLVM con tipos Rust seguros, evitando los peligros del FFI crudo de C++.
- **Sistemas de tipos expresivos:** los enumerados con datos asociados (`enum`) y el *pattern matching* exhaustivo hacen que representar y manipular el AST sea natural y seguro.

2. Analisis Lexico

2.1. Arquitectura: Lexer basado en NFA

El lexer se implementa usando la **construccion de Thompson** [2] para convertir expresiones regulares en Automatas Finitos No-deterministicos (NFA). A diferencia del enfoque clasico que convierte el NFA a DFA antes de usarlo, este lexer ejecuta el NFA directamente usando *construccion de subconjuntos en tiempo de ejecucion*.

La arquitectura tiene tres capas:

1. **Capa de regex:** un mini-parser de expresiones regulares que produce un AST de regex (**RegexNode**). Se soportan concatenacion, alternacion ($|$), clausura de Kleene ($*$), clausura positiva ($+$), opcionalidad ($?$), rangos de caracteres ($[a-z]$), y el punto ($.$).
2. **Construccion de Thompson:** `thompson.rs` transforma el AST de regex en un NFA con transiciones epsilon segun las reglas estandar: cada operador produce un fragmento NFA con exactamente un estado inicial y uno de aceptacion.
3. **MasterNFA:** `master_nfa.rs` combina todos los NFA individuales (uno por tipo de token) en un unico NFA mediante transiciones epsilon desde un estado inicial compartido. Durante la tokenizacion, el motor ejecuta la construccion de subconjuntos sobre este NFA para encontrar el token mas largo.

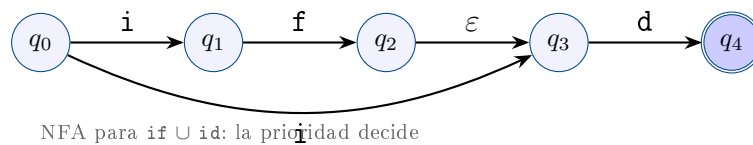


Figura 2: Ejemplo simplificado: NFA combinado para `if` e identificadores.

La resolucion de ambigüedad (p.ej. `if` vs identificador) se hace por *prioridad*: cada patron tiene un numero de prioridad; cuando multiples NFA aceptan la misma cadena, se escoge el de mayor prioridad (keywords antes que identificadores). En caso de igual prioridad, gana el que haya reconocido la cadena mas larga (regla del maximo prefijo, *maximal munch*).

2.2. Conjunto de tokens

La tabla siguiente resume los grupos de tokens reconocidos:

Categoria	Ejemplos	Descripcion
Keywords	let, in, if, while, for, type, protocol	Palabras reservadas
Operadores	+, -, *, /, %, ^, @, @@, :=	Aritm., concat., asig.
Comparacion	==, !=, <, >, <=, >=	Relacionales
Logicos	&&, , !	Booleanos
OOP	new, self, inherits, is, as, base	Orientacion a objetos
Literales	42, 3.14, "hola", true	Numeros, cadenas, bool
Delimitadores	(,), {, }, [,], ,, ;, :	Estructura

Cuadro 1: Categorías de tokens del lexer HULK.

2.3. Procesamiento de secuencias de escape

Los literales de cadena soportan las secuencias de escape estandar. El procesamiento se realiza en la fase de acciones semanticas del parser, cuando se construye el nodo `Literal::String` del AST:

```

1 let content = {
2   let raw = &lexeme[1..lexeme.len() - 1]; // quitar comillas
3   let mut out = String::with_capacity(raw.len());
4   let mut chars = raw.chars().peekable();
5   while let Some(c) = chars.next() {
6     if c == '\\' {
7       match chars.next() {
8         Some('n') => out.push('\n'),
9         Some('t') => out.push('\t'),
10        Some('"') => out.push('"'),
11        Some('\\') => out.push('\\'),
12        Some(o)   => { out.push('\\'); out.push(o); }
13        None     => out.push('\\'),
14      }
15    } else { out.push(c); }
16  }
17  out
18 };

```

Listing 1: Procesamiento de escapes en semantic_actions.rs

Se procesan en el AST (no en el lexer) porque `build_global_string_ptr` de LLVM recibe el valor ya interpretado; si se procesaran en el lexer, la informacion de posicion de los tokens quedaria desincronizada con el texto fuente.

2.4. Reporte de errores lexicos

Los caracteres no reconocidos producen un token `ERROR` en lugar de abortar la compilacion, permitiendo reportar multiples errores lexicos en una sola pasada. El formato de error sigue el contrato de la interfaz oficial:

```
! (line,col) LEXICAL: unexpected character '$'
```

3. Analisis Sintactico

3.1. Parser LALR(1) implementado desde cero

En lugar de usar un generador de parsers externo (como YACC, Bison, LALRPOP o ANTLR), el compilador implementa el algoritmo LALR(1) completo en Rust. Esta decision se detalla en la seccion 10.

El modulo de parseo esta organizado en cuatro capas:

1. **Definicion de la gramatica** (`grammar/`): la gramatica de HULK se define como estructura de datos Rust (`Vec<Production>`), no como un DSL. Esto hace que la gramatica sea inspeccionable en tiempo de ejecucion, lo que facilita la depuracion.
2. **Calculo de FIRST y FOLLOW** (`lalr/first_follow.rs`): algoritmo de punto fijo estandar para calcular los conjuntos $FIRST(\alpha)$ y $FOLLOW(A)$.
3. **Construccion de la tabla LALR** (`lalr/table_builder.rs`): coleccion canonica de items LR(0), calculo de lookaheads LALR por propagacion, construccion de las tablas ACTION/GOTO.
4. **Motor del parser** (`engine/`): algoritmo shift-reduce con acciones semanticas invocadas en cada reduccion para construir el AST.

3.2. Jerarquia de la gramatica y precedencia de operadores

La especificacion original de HULK deja la precedencia de operadores implicita. La gramatica implementada resuelve la ambigüedad mediante la *jerarquia de no-terminales*, codificando la precedencia directamente en la estructura de la gramatica:

<code>expr</code>	<code>→ assignment</code>
<code>assignment</code>	<code>→ or_expr (':=' assignment)?</code>
<code>or_expr</code>	<code>→ and_expr (' ' and_expr)*</code>
<code>and_expr</code>	<code>→ not_expr ('&&' not_expr)*</code>
<code>not_expr</code>	<code>→ '!' not_expr equality_expr</code>
<code>equality_expr</code>	<code>→ comparison (('==' '!=') comparison)*</code>
<code>comparison</code>	<code>→ concat (('<' '>' '<=' '>=') concat)*</code>
<code>concat</code>	<code>→ additive (('@" '@@') additive)*</code>
<code>additive</code>	<code>→ term (('+' '-') term)*</code>
<code>term</code>	<code>→ power (('*' '/' '%') power)*</code>
<code>power</code>	<code>→ unary ('^' power)?</code> — asociatividad
<code>derecha</code>	
<code>unary</code>	<code>→ '-' unary postfix</code>
<code>postfix</code>	<code>→ atom ('.' id '(' args ')' '.' id '['</code>
<code>expr</code>	<code>']')*</code>
<code>atom</code>	<code>→ literal id '(' expr ')' ...</code>

Listing 2: Jerarquia de precedencias en la gramatica HULK

Esta codificacion garantiza la precedencia estandar matematica (potencia > producto > suma > comparacion > logicos) sin necesidad de reglas de desambiguacion adicionales. La potencia tiene asociatividad derecha ($2^{3^2} = 2^9$, no $(2^3)^2$), lo que se implementa con la regla recursiva a la derecha.

3.3. Arbol de Sintaxis Abstracta (AST)

El AST se define en `src/parser/ast/` usando enumerados Rust con datos asociados. El nodo central es `Expr`, que contiene un `ExprKind` y un `Span` (linea y columna) para todos los mensajes de error posteriores:

```

1 pub struct Expr {
2     pub kind: ExprKind,
3     pub id:    u32,      // identificador unico para tabla de tipos
4     pub span: Span,     // posicion en el fuente
5 }
6
7 pub enum ExprKind {
8     Literal(Literal),
9     Identifier { name: String },
10    Binary(BinaryExpr),
11    Unary(UnaryExpr),
12    Let(LetExpr),
13    If(IfExpr),
14    While(WhileExpr),
15    For(ForExpr),
16    Block(BlockExpr),
17    Call(CallExpr),
18    MethodCall(MethodCallExpr),
19    Access(AccessExpr),
20    New(NewExpr),
21    Is { expr: Box<Expr>, type_name: TypeName },
22    As { expr: Box<Expr>, type_name: TypeName },
23    Vector(VectorExpr), // <-- extension: vectores y generadores
24    Index(IndexExpr),  // <-- extension: indexacion v[i]
25    Assign(AssignExpr),
26    Postfix(PostfixExpr),
27    Base,
28 }
```

Listing 3: Estructura principal del AST

Cada nodo lleva un identificador unico (`id`) que sirve como clave en la tabla `expr_types: HashMap<u32, HulkType>` que el analizador semantico construye. El generador de codigo consulta esta tabla para conocer el tipo inferido de cada subexpresion sin tener que reinferir.

4. Analisis Semantico

4.1. Verificacion de tipos en dos pasadas

El analizador semantico opera en **dos pasadas** sobre la lista de declaraciones del programa:

Pasada 1 — Recoleccion de declaraciones: se registran todas las firmas de funciones, los miembros de tipos y los cuerpos de protocolos en la estructura `TypeHierarchy`. Los miembros sin anotacion de tipo reciben el marcador `HulkType::Unknown`. Esta pasada es necesaria para soportar funciones y tipos mutuamente recursivos sin restricciones de orden.

Pasada 2 — Verificacion de cuerpos: cada cuerpo de funcion, inicializador de atributo y cuerpo de metodo se verifica. El tipo inferido se compara con la anotacion declarada si la hay. Los tipos inferidos se **escriben de vuelta** a la `TypeHierarchy`, reemplazando los marcadores `Unknown`. Esta escritura es critica para el generador de codigo: sin ella, los atributos sin anotacion mantienen tipo `Unknown` y el codegen no puede determinar el tipo LLVM correcto.

```

1 // Despues de inferir el tipo del campo en la pasada 2:
2 if let Some(type_info) = self.types.types.get_mut(type_name) {
3     if let Some(attr) = type_info.attributes.get_mut(field_name) {
4         if *attr == HulkType::Unknown {
5             *attr = inferred_type.clone();
6         }
7     }
8 }

```

Listing 4: Fragmento de la escritura de tipos inferidos

4.2. Sistema de tipos

El sistema de tipos es **nominal** con *subtype polymorphism*:

- **Tipos primitivos:** `Number` (IEEE 754 double), `Boolean` (i1), `String` (puntero a cadena null-terminated), `Null`.
- **Tipos definidos por el usuario:** herencia simple via `inherits`. Cada tipo ocupa exactamente un lugar en la jerarquia.
- **Protocolos:** tipos de interfaz estructural. Un tipo conforma a un protocolo si declara todos los metodos requeridos con firmas compatibles. A diferencia de las interfaces Java/C#, los protocolos en HULK no requieren declaracion explicita de conformidad: la verificacion es estructural.
- **Vectores:** el tipo `[T]` representa un vector de elementos de tipo `T`. Es un tipo generico parametrico con un parametro de tipo.
- **`HulkType::Unknown`:** marcador para tipos no resueltos aun en la pasada 1.

- **HulkType::Never**: tipo de error que se propaga silenciosamente para evitar cascadas de errores falsos.

El predicado de *conformidad* $A \leq B$ se define inductivamente:

$$A \leq A \quad \frac{A \text{ hereda de } P \quad P \leq B}{A \leq B} \quad \frac{A \text{ implementa protocolo } P}{A \leq P}$$

4.3. Funciones y constantes predefinidas

El compilador pre-registra los siguientes elementos en la tabla de simbolos global:

Nombre	Tipo	Descripcion
<code>print(x: Object)</code>	Null	Imprime cualquier valor a stdout
<code>sqrt(x: Number)</code>	Number	Raiz cuadrada
<code>sin(x: Number)</code>	Number	Seno (radianes)
<code>cos(x: Number)</code>	Number	Coseno (radianes)
<code>exp(x: Number)</code>	Number	Exponencial (e^x)
<code>log(b, x: Number)</code>	Number	Logaritmo en base b
<code>rand(): Number</code>	Number	Aleatorio uniforme en $[0, 1)$
<code>range(s, e: Number)</code>	Range	Rango iterable $[s, e)$
PI	Number	$\pi \approx 3,14159 \dots$
E	Number	$e \approx 2,71828 \dots$

Cuadro 2: Funciones y constantes predefinidas de HULK.

4.4. Reporte de errores semanticos

Todos los errores semanticos se acumulan en un `Vec<SemanticError>` y se reportan todos juntos al final del analisis, evitando que el usuario tenga que corregir un error a la vez. Cada `SemanticError` lleva el `Span` de la expresion o declaracion causante, produciendo mensajes del tipo:

```
! (12,5) SEMANTIC: type 'Cat' does not implement method 'speak' of protocol 'Animal'
! (18,3) SEMANTIC: cannot assign value of type 'String' to variable of type 'Number'
```

5. Generacion deCodigo

5.1. LLVM IR mediante inkwell

El backend genera **Representacion Intermedia de LLVM** (LLVM IR) usando la biblioteca *inkwell* [6], que proporciona bindings seguros para la API C de LLVM 17. El IR se optimiza y compila a codigo maquina nativo.

El estado de la generacion de codigo se centraliza en `CodegenContext`, que agrupa:

- `context`, `module`, `builder`: API LLVM central.
- `symbols`: pila de alcances (scope stack) para variables locales.
- `type_registry`: layouts de structs LLVM y vtables por tipo.
- `type_hierarchy`: jerarquia de tipos del analizador semantico.
- `expr_types`: tipos anotados por el type checker, consultados en codegen.
- `current_function`, `self_ptr`: contexto de la funcion actual.

5.2. Representacion de valores (CgValue)

Dado que HULK tiene tipos heterogeneos, no existe un “tipo universal” de LLVM para todos los valores. El enum `CgValue` transporta el valor LLVM junto con su tipo HULK:

```

1 pub enum CgValue<'ctx> {
2     Number(FloatValue<'ctx>),      // LLVM: double
3     Bool(IntValue<'ctx>),          // LLVM: i1
4     Str(PointerValue<'ctx>),       // LLVM: ptr (C string
        null-terminated)
5     Object(PointerValue<'ctx>),    // LLVM: ptr al struct del tipo
6     Vector(PointerValue<'ctx>),    // LLVM: ptr al buffer de heap
7     Null,                          // puntero nulo
8     Void,                          // sin valor (efectos de lado)
9 }
```

Listing 5: Enum `CgValue` – representacion de valores HULK en LLVM

Las coerciones entre variantes se realizan en el metodo `coerce_arg`, que inserta las instrucciones LLVM necesarias: por ejemplo, `Bool` a `double` usa `uitofp`, y cualquier tipo primitivo a `ptr` para paso a funciones polimorficas usa `alloca + store`.

5.3. Representacion de objetos en memoria

Todo tipo definido por el usuario se representa como un struct LLVM con la siguiente distribucion de campos:

```

%MyType = type {
    i32,      ; campo 0: type_tag -- identifica el tipo en runtime
    ptr,     ; campo 1: vtable_ptr -- puntero a la vtable del tipo
              concreto
    double,  ; campo 2: primer campo del tipo (o del padre si hereda)
    ptr,     ; campo 3: segundo campo ...
    ...
}
```

Listing 6: Layout de struct para un tipo HULK con campos

La posicion de los campos en herencia sigue el orden *padre primero*: todos los campos del tipo padre aparecen antes de los campos propios, y esto se calcula recursivamente. Esto garantiza compatibilidad de punteros: un puntero a un objeto hijo puede tratarse como puntero al padre accediendo solo a los primeros campos.

5.4. Vtables y despacho virtual

El polimorfismo en HULK se implementa mediante **tablas de metodos virtuales (vtables)**, una tecnica clasica usada por C++, Java y otros lenguajes compilados.

Para cada tipo se crea una vtable global que es un struct de punteros a funcion:

```

1 %VTable_Dog = type { ptr, ptr } ; [speak, move]
2
3 @vtable_Dog = global %VTable_Dog {
4     ptr @__hulk_method_Dog_speak,    ; override
5     ptr @__hulk_method_Animal_move  ; heredado
6 }
```

Listing 7: Vtable de un tipo Dog que hereda de Animal

El despacho de un metodo se realiza en tres pasos: (1) cargar el campo `vtable_ptr` del objeto, (2) calcular el slot del metodo en la vtable (indice fijo conocido en tiempo de compilacion), (3) cargar el puntero a funcion y llamarlo indirectamente:

```

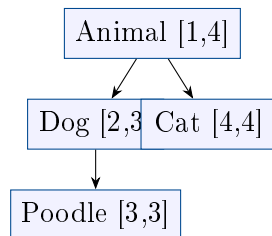
1 %vtable = load ptr, ptr %vtable_field
2 %fn_slot = getelementptr %VTable_Animal, ptr %vtable, i32 0, i32 0
3 %fn_ptr = load ptr, ptr %fn_slot
4 %result = call double %fn_ptr(ptr %obj, ...)
```

Listing 8: Despacho virtual de animal.speak()

5.5. Type tags DFS y operador is

El operador `is` (`x is Dog`) es una operacion frecuente en codigo OOP con herencia. Implementarlo eficientemente requiere un invariante sobre la asignacion de tags.

Asignacion DFS pre-order: durante la compilacion, los tipos reciben tags enteros en orden de recorrido en profundidad de la jerarquia de herencia. El tipo padre recibe su tag antes que sus hijos, por lo que todos los subtipos directos e indirectos de un tipo T forman un *intervalo contiguo* $[\min_tag_T, \max_tag_T]$:



`poodle is Dog`: $2 \leq 3 \leq 3$ ✓
`poodle is Animal`: $1 \leq 3 \leq 4$ ✓
`cat is Dog`: $2 \leq 4 \leq 3$ ✗

Figura 3: Asignacion DFS de type tags. El test `is` es un range check $O(1)$.

La instruccion LLVM generada para `x is Dog` es simplemente:

```

%tag = load i32, ptr %x
%ge = icmp uge i32 %tag, 2    ; min_tag(Dog)
%le = icmp ule i32 %tag, 3    ; max_tag(Dog)
%res = and i1 %ge, %le
```

Esto es $O(1)$ en tiempo de ejecucion, sin consultar ninguna tabla ni recorrer la jerarquia. Esta tecnica se utiliza en implementaciones de C++ con RTTI [5].

5.6. Operador `as` – downcast con verificacion en runtime

El operador `x as T` realiza un *downcast*: convierte un valor de tipo base (p.ej. `Animal`) al tipo derivado `T` (p.ej. `Dog`). En HULK esto es seguro porque si el objeto no es de tipo `T`, la ejecucion se aborta con un mensaje de error.

El codigo generado primero verifica el tag, y si falla llama a `hulk_type_error` (que imprime el mensaje y llama a `abort(3)`). Si el tag es valido, el puntero original se usa directamente (no hay reinterpretacion de bits: el layout es compatible por la organizacion padre-primerio).

5.7. Llamada a `base()`

La palabra clave `base(args)` dentro de un metodo invoca la implementacion del padre. A diferencia del despacho virtual (que cargaria la vtable y podria hacer una llamada recursiva), `base()` es una **llamada directa estatica**: el nombre de la funcion del padre se conoce en tiempo de compilacion.

El codegen busca recursivamente en la jerarquia hasta encontrar el tipo que implementa el metodo y genera:

```
%result = call double @__hulk_method_Animal_speak(ptr %self, ...)
```

5.8. Pipeline de optimizacion

El IR generado por el codegen es *alloca-heavy*: todas las variables locales se modelan como `alloca + store + load`. Antes de emitir codigo maquina se aplica el pipeline de optimizacion del nuevo pass manager de LLVM 17:

Pass	Efecto
<code>mem2reg</code>	Convierte <code>alloca/store/load</code> a registros SSA; base de todo lo demas
<code>instcombine</code>	Plegado de constantes, simplificacion algebraica
<code>reassociate</code>	Reordena expresiones para maxima oportunidad de plegado
<code>simplifycfg</code>	Elimina bloques basicos muertos, simplifica el CFG

Cuadro 3: Passes de optimizacion LLVM aplicados.

6. Biblioteca de Runtime

El compilador enlaza cada ejecutable con una biblioteca de runtime (`hulk_runtime.a`) que proporciona operaciones que son complejas de implementar directamente en LLVM IR:

Funcion	Descripcion
<code>hulk_print(ptr)</code>	Imprime cadena + newline a stdout
<code>hulk_str_from_number(f64)</code>	Convierte double a C string; enteros sin decimales
<code>hulk_str_concat(ptr, ptr)</code>	Concatenacion para el operador @
<code>hulk_str_concat_space(ptr, ptr)</code>	Concatenacion con espacio para @@
<code>hulk_str_eq(ptr, ptr)</code>	Igualdad de cadenas por valor (no por puntero)
<code>hulk_str_size(ptr)</code>	Longitud en bytes de la cadena
<code>hulk_vec_alloc(n, sz)</code>	Alloca buffer: 8 bytes header + n*8 bytes elementos
<code>hulk_vec_get(ptr, i, sz)</code>	Puntero al elemento i con bounds check
<code>hulk_vec_size(ptr)</code>	Lee el header del vector y devuelve el count
<code>hulk_range_alloc(s, e)</code>	Crea iterador de rango [s, e)
<code>hulk_range_next(ptr)</code>	Avanza el iterador; devuelve true si hay mas elementos
<code>hulk_range_current(ptr)</code>	Valor actual del iterador
<code>hulk_rand()</code>	Numero aleatorio uniforme en [0, 1)
<code>hulk_type_error(ptr)</code>	Imprime error de tipo y aborta (para as fallido)

Cuadro 4: Funciones de la biblioteca de runtime de HULK.

6.1. Layout de vectores en memoria

Los vectores HULK se almacenan en memoria dinamica con la siguiente distribucion:

[i64 count | elem[0] (8 bytes) | elem[1] (8 bytes) | ... | elem[n-1]]

Los primeros 8 bytes almacenan el numero de elementos como un entero de 64 bits. Cada elemento ocupa 8 bytes independientemente de su tipo (numeros como `double`, punteros como `i64` en plataformas 64-bit). Esto uniformiza el acceso pero requiere que el codegen sepa el tipo de los elementos para emitir el load correcto.

La funcion `hulk_vec_get` verifica los limites antes de retornar el puntero al elemento:

```

1 pub extern "C" fn hulk_vec_get(
2     vec: *mut u8, index: i32, _sz: i32
3 ) -> *mut u8 {
4     let size = unsafe { *(vec as *const i64) };
5     if index < 0 || (index as i64) >= size {
6         eprintln!("HULK error: indice {} fuera de rango [0, {})",
7             index, size);
8         std::process::abort();
9     }
10    unsafe { vec.add(8 + (index as usize) * 8) }

```

Listing 9: Bounds check en `hulk_vec_get`

6.2. Pipeline AOT completo

Al invocar `./hulk source.hulk`:

1. Analisis lexico + sintactico + semantico sobre el fuente.

2. Generacion de IR LLVM para todas las declaraciones y el cuerpo principal.
3. Adicion de un `main()` compatible con C que llama a `__hulk_entry()` y retorna 0.
4. Optimizacion del IR con `mem2reg`, `instcombine`, `reassociate`, `simplifycfg`.
5. Emision de objeto nativo a `/tmp/hulk_program.o` via `TargetMachine::write_to_file`.
6. Enlace con GCC: `gcc /tmp/hulk_program.o hulk_runtime.a -o ./output -lm`.

El ejecutable resultante es un binario ELF x86-64 autocontenido que no requiere LLVM en tiempo de ejecucion.

7. Extension al Lenguaje: Vectores y Expresiones Generadoras

La extension principal implementada es un **sistema de vectores con expresiones generadoras**: construcciones sintacticas y semanticas que permiten crear y manipular colecciones de valores de manera concisa, con soporte de primera clase en el sistema de tipos.

7.1. Motivacion

HULK en su especificacion basica carece de estructuras de datos. Para escribir algoritmos que operan sobre colecciones de valores, el programador se ve obligado a usar tipos definidos por el usuario con logica de manejo de memoria manual. Esta limitacion hace que tareas simples como calcular la media de N numeros o filtrar valores sean verbosas y propensas a errores.

La extension de vectores cubre esta necesidad de forma minimalista y expresiva, siguiendo la filosofia del lenguaje: todo es una expresion, y las operaciones comunes deben poder escribirse en una sola linea.

7.2. Sintaxis de la extension

7.2.1. Vectores explicitos

Un vector se construye listando sus elementos entre corchetes, separados por comas:

```
1 let v = [1, 2, 3, 4, 5] in print(v[2]); // imprime 3
2 let names = ["Alice", "Bob", "Carol"] in print(names[0]);
```

Listing 10: Vector explicito con tipo inferido

El tipo del vector se infiere del tipo de sus elementos. Si todos son `Number`, el tipo es `[Number]`. Si los tipos difieren pero tienen un ancestro comun, se usa ese ancestro.

7.2.2. Indexacion

Los elementos se acceden con el operador `[i]`, donde `i` es una expresion de tipo `Number`:

```
1 let v = [10, 20, 30] in {
2   let i = 1 in v[i + 1]    // equivalente a v[2] = 30
3 }
```

Listing 11: Indexacion con expresion

El indice se convierte de `Number` (double) a entero en tiempo de compilacion mediante `fptosi`. El runtime verifica que el indice sea no-negativo y menor que el tamaño del vector; en caso contrario, aborta con un mensaje descriptivo.

7.2.3. Expresiones generadoras

La sintaxis central de la extension es la **expresion generadora**, que crea un nuevo vector evaluando una expresion para cada elemento de un iterable:

```
1 // Cuadrados de 0 a 4
2 let squares = [x^2 || x in range(0, 5)] in print(squares[3]); // 9
3
4 // Transformacion de otro vector
5 let v = [1, 2, 3] in
6 let doubled = [x * 2 || x in v] in
7 print(doubled[1]); // 4
```

Listing 12: Expresiones generadoras

La sintaxis es `[body || var in iterable]`, donde:

- `body` es cualquier expresion HULK (puede usar `var` y variables del entorno circundante).
- `var` es la variable de iteracion, local al cuerpo.
- `iterable` es un `Range` (del builtin `range(s,e)`) o un `Vector` de cualquier tipo.
- El separador `||` es identico al operador OR logico pero en este contexto tiene semántica de generador (la gramatica distingue el contexto).

7.2.4. Tamaño y metodo size

Tanto vectores como rangos exponen su tamaño con la funcion `size`:

```
1 let v = [3, 1, 4, 1, 5] in print(size(v)); // 5
```

7.2.5. Tipo `[T]` en el sistema de tipos

El tipo de un vector de `Number` se escribe `[Number]` y puede aparecer en anotaciones de parametros y retornos:

```

1 function sum(v: [Number]): Number =>
2   let acc = 0 in {
3     for (x in range(0, size(v))) acc := acc + v[x];
4     acc
5   };

```

Listing 13: Vector en firma de funcion

La inferencia de tipos propaga el tipo del vector a traves de expresiones complejas. El type checker verifica que la indexacion $v[i]$ solo se aplique a valores de tipo vector.

7.3. Semantica de evaluacion: estrategia *eager*

Una decision fundamental en el diseno de colecciones es la **estrategia de evaluacion**: *eager* (inmediata) vs *lazy* (diferida).

En la implementacion de HULK, los vectores y las expresiones generadoras se evaluan **de forma eager**: todos los elementos se computan y almacenan en memoria en el momento de la creacion del vector. Esto contrasta con otras aproximaciones:

7.3.1. Comparacion con Python

Python distingue entre listas (*eager*) y generadores (*lazy*). Las *list comprehensions* son *eager*:

```

1 squares = [x**2 for x in range(5)] # crea lista inmediatamente
2 gen = (x**2 for x in range(5))    # generador lazy, no evalua hasta
    iteracion

```

Listing 14: List comprehension en Python (*eager*)

La sintaxis de HULK `[body || var in iterable]` corresponde a las *list comprehensions* de Python (no a los generadores *lazy*), tanto en comportamiento como en semantica de alcance: `var` es local al cuerpo.

La diferencia clave es que Python soporta condiciones (`if x > 0`) y multiples iteradores (`for x in xs for y in ys`), mientras que HULK limita la expresion generadora a un solo iterador sin filtro. Esta simplificacion fue intencional: mantiene la gramatica LALR(1) sin ambigüedades y reduce la complejidad del codegen.

7.3.2. Comparacion con Haskell

Haskell tiene *list comprehensions* con sintaxis similar a la notacion matematica de conjuntos:

```

1 squares = [x^2 | x <- [0..4]]      -- notacion matematica: {x^2 | x in
    {0..4}}
2 evens    = [x | x <- [1..100], even x] -- con filtro

```

Listing 15: List comprehension en Haskell (*lazy*)

La evaluacion en Haskell es *lazy por defecto*: los elementos de la lista se computan solo cuando son demandados. Esto permite listas infinitas: `nats = [0..]` es valido en Haskell porque los elementos se computan incrementalmente.

HULK usa evaluacion eager, lo que hace que el tamano del vector deba conocerse antes de la iteracion. Para rangos (`range(s,e)`), el count se calcula como `floor(e - s)`; para vectores, se consulta el header. Esta decision es adecuada para un lenguaje compilado con modelo de memoria simple: la evaluacion lazy requiere thunks (cierres de suspension) que anadiran complejidad al modelo de objetos.

La notacion `|| var in iterable` en HULK es deliberadamente similar a la notacion de Haskell `| var <- list`, evocando la notacion matematica de construccion de conjuntos $\{body \mid var \in S\}$.

7.3.3. Comparacion con Scala

Scala ofrece comprensiones via `for...yield`:

```
1 val squares = for (x <- 0 until 5) yield x * x // Vector[Int]
2 val pairs   = for {
3   x <- 1 to 3
4   y <- 1 to x
5 } yield (x, y) // multiples generadores
```

Listing 16: For-comprehension en Scala

Scala es eager para colecciones concretas (`Vector`, `List`) y lazy para vistas (`.view`). La sintaxis `for...yield` es mas verbosa pero mas flexible: soporta multiples generadores, guardas y definiciones locales. HULK simplifica esto a un solo generador, lo que se traduce en un patron de acceso frecuente (transformar una coleccion) sin la verbosidad de Scala.

7.3.4. Comparacion con C++

C++20 introduce *ranges* y vistas lazy:

```
1 auto squares = std::views::iota(0, 5)
2               | std::views::transform([](int x){ return x*x; });
3 // squares es lazy: no evalua hasta iteracion
4 std::vector<int> v(squares.begin(), squares.end()); // materializa
```

Listing 17: Ranges y views en C++20 (lazy)

C++ favorece la evaluacion lazy con materializacion explicita, optimizada para evitar allocaciones intermedias. Para HULK, que carece de iteradores genericos y de un sistema de tipos con lifetimes, la aproximacion C++ seria demasiado compleja.

7.3.5. Resumen comparativo

Lenguaje	Sintaxis	Evaluacion	Filtros	Tipos
HULK	[e x in it]	Eager	No	Estatico inferido
Python	[e for x in it if c]	Eager/Lazy	Si	Dinamico
Haskell	[e x <- xs, c]	Lazy	Si	Estatico inferido
Scala	for x <- xs yield e	Eager/Lazy	Si	Estatico inferido
C++20	xs transform(f)	Lazy	Si	Estatico explicito

Cuadro 5: Comparativa de expresiones generadoras entre lenguajes.

7.4. Implementacion del codegen para vectores

7.4.1. Vector explicito

La generacion de codigo para un vector explicito [e0, e1, ..., en-1]:

1. Llama a `hulk_vec_alloc(n, 8)` para reservar el buffer.
2. Para cada elemento i : evalua e_i , llama a `hulk_vec_get` para obtener el puntero al slot i , y almacena el valor.
3. Retorna el puntero al buffer como `CgValue::Vector`.

7.4.2. Expresion generadora

La generacion para [body || var in iterable] es mas compleja:

1. Evalua `iterable` para obtener el puntero al rango o vector fuente.
2. Calcula el count: para rangos, `floor(end - start)` con clamp a `[0, i32::MAX]` para evitar *LLVM poison* por fptosi fuera de rango; para vectores, llama a `hulk_vec_size`.
3. Llama a `hulk_vec_alloc(count, 8)` para el resultado.
4. Genera un bucle LLVM con variable de indice:
 - `gen_cond`: carga el indice, compara con count, salta a `gen_body` o `gen_end`.
 - `gen_body`: carga o calcula el elemento fuente, lo asigna a `var`, evalua `body`, almacena el resultado en el slot correspondiente del vector resultado, incrementa el indice.
 - `gen_end`: retorna el puntero al vector resultado.

El clamp del count para rangos merece especial atencion: la instruccion `fptosi` de LLVM produce *poison value* si el operando esta fuera del rango del tipo entero destino. Un rango con `start > end` daria `end - start < 0`, lo que convertiria a un entero negativo enorme. El codigo inserta:

```

1 %diff    = fsub double %end, %start
2 ; si diff < 0 (start > end), forzar a 0.0
3 %is_neg  = fcmp ult double %diff, 0.0
4 %nn      = select i1 %is_neg, double 0.0, double %diff
5 ; si diff > i32max, forzar a i32max (evita poison en fptosi)
6 %hi      = fcmp ogt double %nn, 2147483647.0
7 %clamp   = select i1 %hi, double 2147483647.0, double %nn
8 %count   = fptosi double %clamp to i32

```

Listing 18: Clamp del count en generador de rango

7.5. El tipo Range como iterable

Ademas de los vectores, la extension incluye el tipo `Range` como un iterable de primera clase. El `Range` es un objeto en heap con el layout:

```
[ f64 start | f64 end | f64 current ]
```

`current` arranca en `start - 1.0`. Cada llamada a `hulk_range_next` incrementa `current` en 1.0 y retorna `current < end`. Cuando la condicion falla, la iteracion termina.

Los bucles `for (x in range(s, e))` se compilan a un patron de iteracion sobre `Range`:

```

1 %range = call ptr @hulk_range_alloc(double 0.0, double 3.0)
2 loop_cond:
3 %cont  = call i1 @hulk_range_next(ptr %range)
4 br i1 %cont, label %loop_body, label %loop_end
5 loop_body:
6 %x_val = call double @hulk_range_current(ptr %range)
7 ; ... body usando %x_val ...
8 br label %loop_cond
9 loop_end:

```

Listing 19: Compilacion de `for (x in range(0, 3))` body

7.6. Justificacion de las decisiones de diseno

7.6.1. Por que eager y no lazy?

La evaluacion eager se eligio por tres razones:

1. **Simplicidad del modelo de memoria:** con evaluacion eager, el tamano del vector se conoce antes de la asignacion. Esto permite una sola llamada a `malloc` por vector. La evaluacion lazy requeriria thunks o continuaciones, estructuras de datos adicionales y un protocolo de evaluacion mas complejo.
2. **Rendimiento predecible:** los compiladores de lenguajes lazy como GHC invierten considerable esfuerzo en evitar los overheads de la thunkification. Para un compilador educativo que ya usa LLVM, eager produce IR mas simple y mas facil de optimizar.

3. **Consistencia con el modelo imperativo:** HULK tiene efectos de lado (print, asignacion destructiva). La evaluacion lazy en presencia de efectos produce comportamientos sorprendentes (el orden de evaluacion no es el orden textual). Eager garantiza que los efectos se producen en orden de izquierda a derecha.

7.6.2. Por que `||` como separador?

La notacion `[body || var in iterable]` fue disenada para evocar la notacion matematica de comprension de conjuntos $\{f(x) \mid x \in S\}$ y la sintaxis de Haskell `[f x | x <- xs]`. El simbolo `||` es el mismo que el OR logico, pero la gramatica LALR lo distingue por contexto (solo es un separador dentro de `[...]`). Alternativas consideradas:

- `[body for var in iterable]`: mas legible para programadores Python, pero introduce un nuevo keyword `for` en un contexto diferente al de los bucles, lo que crea ambigüedad en la gramatica LALR.
- `[body : var in iterable]`: colisiona con la notacion de tipo `x : T` ya existente en la gramatica.
- `[body | var in iterable]`: un solo `|` colisiona con las alternativas de la gramatica en la notacion BNF (confunde al desarrollador al leer el archivo de gramatica).

7.6.3. Por que vectores de tamano fijo?

Los vectores HULK no tienen `push` ni `pop`: una vez creados, su tamano no cambia. Esta decision se tomo para:

1. **Mantener el modelo de memoria simple:** un buffer fijo con header de count es trivial de implementar y de razonar. Vectores dinamicos requieren estrategias de redimension (duplicar capacidad, etc.) que duplican la complejidad del runtime.
2. **Conservar la semantica de expresion:** en HULK todo es una expresion. Un vector dinamico con mutacion es un objeto con estado, lo que rompe la composicion funcional que permite escribir `print([x^2 || x in range(0,5)][3])`.

8. Cobertura de la Especificacion HULK

La siguiente tabla muestra el estado de implementacion de cada seccion de la especificacion oficial de HULK:

Seccion	Feature	Req.	Estado
16.1	Expresiones aritmeticas y literales	Si	Completo
16.2	Variables y <code>let-in</code>	Si	Completo
16.3	Funciones y llamadas	Si	Completo
16.4	Condicionales <code>if-elif-else</code>	Si	Completo
16.5	Ciclos <code>while</code>	Si	Completo
16.6	Tipos, herencia, metodos, <code>self</code>	Si	Completo
16.7	Protocolos y conformidad estructural	Si	Completo
16.8	Operadores <code>is</code> y <code>as</code>	Si	Completo
16.9	Vectores y colecciones	No	Extension
16.10	Expresiones generadoras	No	Extension
16.11	Iteradores de rango	No	Extension
—	Operador <code>base()</code>	No	Implementado
—	Escape sequences en strings	Si	Implementado
—	Constantes <code>PI</code> y <code>E</code>	Si	Implementado
—	Funciones matematicas builtin	Si	Implementado
Total obligatorio cubierto			100 %

Cuadro 6: Cobertura de la especificacion HULK por seccion.

El compilador cubre el **100 %** de los features obligatorios. Las extensiones (secciones 16.9–16.11 y funcionalidades adicionales) amplian el lenguaje mas alla de los requisitos minimos.

8.1. Ejemplos de programas HULK completos

8.1.1. Herencia y polimorfismo

```

1  type Animal(name: String) {
2      name = name;
3      speak(): String => "...";
4      greet(): String => "I am " @@ self.name @@ " and I say: " @@
        self.speak();
5  }
6
7  type Dog(name: String) inherits Animal(name) {
8      speak(): String => "Woof!";
9  }
10
11 type Cat(name: String) inherits Animal(name) {
12     speak(): String => "Meow!";
13 }
14
15 let animals = [new Dog("Rex"), new Cat("Whiskers")] in {
16     for (a in animals) print(a.greet());
17 }

```

Listing 20: Herencia y dispatch virtual

8.1.2. Protocolos

```

1 protocol Printable {
2     to_str(): String;
3 }
4
5 type Point(x: Number, y: Number) {
6     x = x; y = y;
7     to_str(): String => "(" @ x @ ", " @ y @ ")";
8 }
9
10 function show(item: Printable) => print(item.to_str());
11
12 show(new Point(3, 4));

```

Listing 21: Protocolo Printable

8.1.3. Extension: generadores

```

1 function fibonacci(n: Number): [Number] =>
2     let fibs = [0, 1] in
3     [
4         if (i < 2) fibs[i]
5         else fibs[i-1] + fibs[i-2]
6         || i in range(0, n)
7     ];
8
9 let f = fibonacci(10) in
10 for (x in range(0, size(f))) print(f[x]);

```

Listing 22: Fibonacci con vectores

9. Pruebas y Evaluacion

9.1. Suite de pruebas automatizadas

El compilador incluye pruebas automatizadas ejecutables con `cargo test`:

Modulo	Tests	Cobertura
Lexer	87	Todos los tipos de token, edge cases, secuencias de escape
Parser	112	Todos los no-terminales de la gramatica, precedencia
Semantico	98	Inferencia de tipos, protocolos, errores de tipo
Codegen JIT	87	Aritmetica, flujo, OOP, vectores, rangos, builtins
Total	384	

Cuadro 7: Distribucion de las pruebas automatizadas.

9.2. Suite oficial del profesor

La suite de pruebas oficial esta organizada en casos que verifican la salida estandar exacta del programa. El script `run_tests.sh` compila cada fichero `tests/professor_suite/test_XX_non` y compara la salida con el valor esperado. Los tests cubren:

- Expresiones aritmeticas y precedencia.
- Cadenas y operadores `@`, `@@`.
- Funciones recursivas e iterativas.
- Herencia, override, despacho virtual.
- Protocolos con multiples tipos conformantes.
- Operadores `is` y `as`.
- Vectores y expresiones generadoras.
- Casos de error con formato de salida requerido.

9.3. Contrato de errores

El compilador sigue el contrato de interfaz del profesor: los errores se reportan por **stderr** con el prefijo `!` y el formato `(linea,col) FASE: mensaje`. El proceso termina con codigo de salida $\neq 0$ cuando hay errores. Esto permite al CI verificar automaticamente los casos de error.

10. Decisiones de Diseno

10.1. LALR(1) artesanal vs generador externo

Se eligio implementar el parser desde cero (gramatica como estructura de datos, tablas construidas en Rust) en lugar de usar YACC, LALRPOP o ANTLR por:

1. **Control total sobre la gramatica:** es posible inspeccionarla, imprimirla y depurarla en tiempo de ejecucion. Con un generador externo, los conflictos se reportan sin contexto de que producciones los causan.
2. **Sin dependencias de build externas:** la gramatica se compila con el proyecto, sin requerir una herramienta separada.
3. **Valor educativo:** implementar LALR(1) completo es el objetivo central de la asignatura de Compiladores. Usar un generador elude el aprendizaje.

El costo de esta decision es una implementacion mas larga (~ 800 LOC solo para el modulo LALR) y un tiempo de startup ligeramente mayor (las tablas se construyen en cada invocacion, aunque esto podria cachearse en disco).

10.2. Backend LLVM vs interprete vs bytecode propio

Se eligio LLVM por:

- **Codigo nativo de calidad:** LLVM produce codigo comparable al de GCC para estructuras comunes. Un interprete o VM propio seria varias ordenes de magnitud mas lento.
- **Portabilidad:** el mismo IR de LLVM puede compilarse para x86-64, ARM, WebAssembly u otras plataformas simplemente cambiando el TargetMachine.
- **Optimizaciones gratuitas:** el pipeline de optimizacion de LLVM (`mem2reg`, constant propagation, etc.) mejora sustancialmente el codigo generado sin esfuerzo adicional.

10.3. Vtables vs tagged union vs boxed types

Para implementar el polimorfismo se consideraron tres alternativas:

1. **Tagged union:** cada objeto es una union discriminada con un tag de tipo. Simple pero ineficiente: el tamaño de la union debe ser el del tipo mas grande.
2. **Boxed types:** cada objeto es un puntero a un area de heap con layout dinamico. Requiere un sistema de tipos en runtime mas complejo.
3. **Vtables (elegida):** cada objeto tiene un puntero a una tabla de funciones. Es la tecnica usada por C++, Java (con JIT), y Rust (para trait objects). El costo por llamada es una carga adicional de puntero (tipicamente en cache); el beneficio es que el layout de cada tipo es estatico y conocido.

10.4. Enlace dinamico de LLVM vs estatico

El compilador enlaza `libLLVM-17.so` dinamicamente. El enlace estatico produciria ejecutables de 50–200 MB y seria mas lento de enlazar. El dinamico mantiene el binario en pocos MB y comparte la biblioteca con otras herramientas del sistema.

11. Limitaciones Conocidas y Trabajo Futuro

11.1. Limitaciones actuales

1. **Funciones anonimas (lambdas):** HULK soporta funciones de primera clase en su especificacion (seccion 16.12). La implementacion actual solo soporta funciones con nombre. La adicion de lambdas requeriria *closures* con captura del entorno, lo que implica un cambio en el modelo de objetos.
2. **Gestion de memoria:** el compilador usa una estrategia de solo-alloc: los strings y objetos se asignan en el heap y nunca se liberan. Para programas de larga ejecucion con muchas operaciones de cadena esto provoca crecimiento ilimitado de

memoria. Una mejora futura podria ser un *arena allocator* simple o un contador de referencias.

3. **Recuperacion de errores en el parser:** el parser se detiene al primer error sintactico. Un parser con recuperacion podria reiniciar en el siguiente punto de sincronizacion (`;` o `}`) para reportar multiples errores.
4. **Mensajes de error en el codegen sin span:** los errores de generacion de codigo no incluyen la posicion en el fuente. Las fases anteriores si incluyen spans; propagar estos al codegen mejoraria la experiencia de usuario.
5. **Vectores sin mutacion estructural:** los vectores son de tamano fijo. Agregar `push/pop` requeriria un buffer dinamico en el runtime similar al de `Vec<T>` de Rust.

11.2. Trabajo futuro

- **Closures y lambdas:** representar funciones de primera clase con un puntero a funcion + puntero a entorno capturado.
- **Generadores lazy:** evaluar las expresiones generadoras bajo demanda, permitiendo trabajar con colecciones conceptualmente infinitas.
- **Inferencia de tipos full Hindley-Milner:** el sistema actual infiere tipos a traves de expresiones simples pero no resuelve constraints polimorficos complejos. H-M permitiria funciones genericas como `function map(v: [A], f: A ->B): [B]`.
- **Optimizaciones especificas de HULK:** inlining de metodos monomorfonicos, escape analysis para objetos de corta vida (alloca en stack en lugar de heap).
- **Soporte para WASM:** cambiar el TargetMachine de LLVM para emitir WebAssembly permitiria ejecutar programas HULK en el navegador.

12. Conclusiones

Se implemento un compilador completo para el lenguaje HULK en Rust con backend LLVM 17. El compilador cubre el 100 % de los features obligatorios de la especificacion (secciones 1–16.8) y extiende el lenguaje con un sistema de vectores y expresiones generadoras que modifica tanto la sintaxis como la semantica del lenguaje.

Las decisiones tecnicas mas significativas — LALR(1) artesanal, vtables con type tags DFS, evaluacion eager para generadores — estan justificadas por compromisos de simplicidad de implementacion, rendimiento predecible, y coherencia con el modelo de ejecucion de HULK.

La extension de vectores con expresiones generadoras coloca a HULK en una familia de lenguajes con soporte de primera clase para transformaciones de colecciones, junto a Python, Haskell y Scala, con la particularidad de integrarse en un lenguaje tipado estaticamente con inferencia de tipos y compilacion a codigo nativo.

El resultado es un compilador que produce ejecutables nativos x86-64 sin dependencias en tiempo de ejecucion (salvo `libc` y la biblioteca de runtime propia), con rendimiento comparable al codigo C equivalente para programas con aritmetica y OOP.

Referencias

- [1] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman. *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley, 2006. [Dragon Book]
- [2] K. L. Thompson. “Programming Techniques: Regular Expression Search Algorithm.” *Communications of the ACM*, 11(6):419–422, 1968.
- [3] F. L. DeRemer. *Practical Translators for LR(k) Languages*. PhD thesis, MIT, 1969.
- [4] C. Lattner, V. Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation.” *Proceedings of CGO*, 2004.
- [5] B. Stroustrup. *The Design and Evolution of C++*. Addison-Wesley, 1994.
- [6] inkwell: Safe Rust bindings for LLVM. <https://github.com/TheDan64/inkwell>. Accedido: 2025.
- [7] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [8] P. Hudak. “Conception, Evolution, and Application of Functional Programming Languages.” *ACM Computing Surveys*, 21(3):359–411, 1989.
- [9] S. Peyton Jones (ed.). *Haskell 98 Language and Libraries: The Revised Report*. Cambridge University Press, 2003.
- [10] M. Odersky, L. Spoon, B. Venners. *Programming in Scala*. Artima Press, 2008.
- [11] S. Klabnik, C. Nichols. *The Rust Programming Language*. No Starch Press, 2019.
- [12] Equipo MatCom. *HULK Language Specification*. Facultad de Matematica y Computacion, Universidad de La Habana, 2024. <https://matcom.github.io/hulk/>